

Motion Planning per Robot Uniciclo con Algoritmo RRT

Questo progetto ha l'obiettivo di estendere l'algoritmo RRT (Rapidly-exploring Random Tree), adattandolo a un robot mobile con vincoli cinematici reali.

RRT standard lavora su modelli puntiformi e si basa su collegamenti lineari tra i nodi, invece in questa versione il robot viene descritto come un corpo poligonale che segue il modello cinematico di tipo uniciclo. Ciò significa che il percorso generato non è composto da segmenti retti, ma da archi di circonferenza che rispettano la direzione del veicolo e il suo raggio minimo di sterzata.

Obiettivi

L'obiettivo principale è trovare un percorso collision-free per un robot mobile non puntiforme, modellato come poligono con orientazione. Il robot si dovrà muovere come farebbe un veicolo uniciclo, evitando movimenti fisicamente impossibili.

L'algoritmo pianifica traiettorie continue, tenendo conto dell'orientazione del robot e dello spazio che occupa intorno a sé.

Logica di Funzionamento

Le fasi principali che ripete l'algoritmo sono:

1. Campionamento: viene scelto un punto casuale nello spazio operativo.
2. Si trova il nodo dell'albero più vicino al punto campionato.
3. Si calcola il movimento (arco di circonferenza) che connette il nodo al nuovo stato, rispettando i vincoli del modello uniciclo.
4. L'intero arco viene verificato per assicurarsi che il robot non collida con gli ostacoli.
5. Se il percorso è valido, il nuovo nodo viene aggiunto all'albero di esplorazione.
6. Periodicamente l'algoritmo tenta di connettersi direttamente all'obiettivo per accelerare la convergenza verso la soluzione.

Struttura del Codice

Il progetto è organizzato in file, ciascuno con un ruolo specifico:

- **robot.py** – definisce la geometria del robot come un poligono convesso (rettangolo) invece che come un punto.
Fornisce funzioni per posizionare e ruotare il robot a partire da una posa (x, y, θ) . Il risultato viene usato per la collision detection e per la visualizzazione del robot durante la simulazione.
- **collision.py** – gestisce la collision detection del robot con gli ostacoli e con i limiti dell'ambiente. Per ogni posa generata durante la propagazione del modello unicycle viene calcolata la footprint del robot e verificata l'intersezione con gli ostacoli usando Shapely. In caso di collisione lungo un arco, il sistema restituisce l'ultima posa sicura per permettere all'RRT di continuare l'esplorazione.

- **unicycle.py** – implementa il modello cinematico dell'uniciclo e calcola gli archi di circonferenza che collegano un nodo dell'albero a un punto campionato. Si determina l'arco tangente alla direzione del robot e successivamente si discretizza nel tempo tramite integrazione del modello. Il risultato è una sequenza di pose lungo l'arco che verranno poi verificate per le collisioni e utilizzate dall'algoritmo RRT.
- **rrt.py** – implementa l'algoritmo RRT esteso per robot unicycle, dove ogni nodo contiene la posa (x, y, θ) e le connessioni sono archi percorribili. L'albero viene espanso tramite campionamento casuale, verifica collisioni e tentativi periodici di connessione al goal. Restituisce l'albero generato e il percorso finale se trovato.
- **server.py** – implementa un server web con FastAPI e WebSocket per visualizzare in tempo reale l'esecuzione dell'algoritmo RRT. Il server avvia l'algoritmo in background, invia progressivamente i nodi generati al browser e permette di avviare o interrompere la simulazione tramite interfaccia web. In questo modo è possibile osservare la crescita dell'albero e il percorso trovato direttamente da una pagina locale.

Esecuzione della Simulazione

Per il corretto funzionamento sono necessarie le seguenti librerie Python:

- numpy – per il calcolo numerico;
- shapely – per la gestione geometrica e rilevamento collisioni;
- fastapi e uvicorn – per l'interfaccia web di simulazione;
- websockets – per la comunicazione in tempo reale.

L'installazione può essere effettuata con: `pip install -r requirements.txt`

Per lanciare la simulazione interattiva e visualizzare il comportamento dell'algoritmo:

`python3 server.py`

Nel browser si può aprire localhost per osservare l'esecuzione del RRT: il robot esplora l'ambiente, evita gli ostacoli e pianifica autonomamente un percorso verso il traguardo.